

Homework 02

Question 1 Answers

Part a

We fix $\mathbf{X}_{\text{all}}$ and μ once, then over 200 reps draw fresh noise, fit models $j = 0, \dots, q$, and record training/test MSE.

```
set.seed(43202)
n <- 100
q <- 20

X_all <- matrix(rnorm(n * q), nrow = n, ncol = q)
beta <- 0.4^sqrt(1:q)
mu <- as.vector(X_all %*% beta)

n_sim <- 200
mse_train <- matrix(NA, n_sim, q + 1)
mse_test <- matrix(NA, n_sim, q + 1)

for (s in 1:n_sim) {
  epsilon <- rnorm(n)
  epsilon_star <- rnorm(n)
  y <- mu + epsilon
  y_star <- mu + epsilon_star

  for (j in 0:q) {
    Xj <- cbind(1, X_all[, seq_len(j), drop = FALSE])
    fit <- lm.fit(x = Xj, y = y)
    mu_hat <- as.vector(Xj %*% fit$coefficients)
    mse_train[s, j + 1] <- mean((y - mu_hat)^2)
    mse_test[s, j + 1] <- mean((y_star - mu_hat)^2)
  }
}

avg_train_mse <- colMeans(mse_train)
avg_test_mse <- colMeans(mse_test)
j_grid <- 0:q

# Training MSE should never increase as predictors are added, within a run.
train_is_monotone <- all(apply(mse_train, 1, function(row) all(diff(row) <= 1e-8)))
print(train_is_monotone)
```

```
[1] TRUE
```

```
plot(
  j_grid, avg_train_mse, type = "l", lwd = 2, col = "#1F5A94",
  ylim = range(avg_train_mse, avg_test_mse),
  xlab = "Number of predictors (j)", ylab = "Average MSE"
)
lines(j_grid, avg_test_mse, lwd = 2, col = "#C84A16")
legend(
  "topright", bty = "n", lwd = 2, col = c("#1F5A94", "#C84A16"),
```

```

legend = c("Average training MSE", "Average test MSE")
)

```

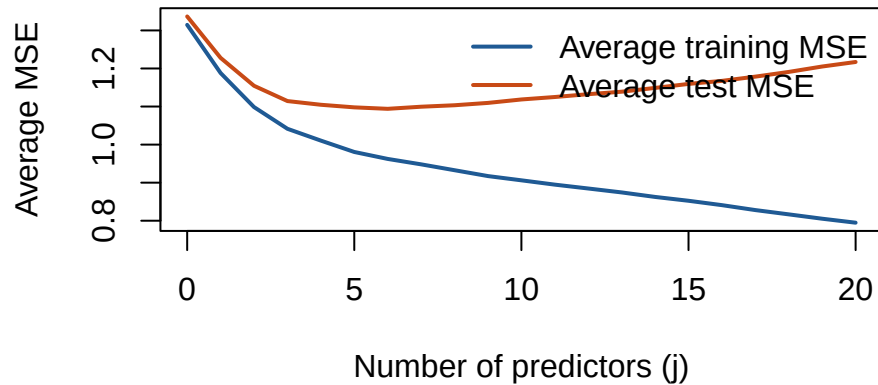


Figure 1: Average training and test MSE across 200 simulations.

`train_is_monotone` is TRUE: training MSE never increases as predictors are added, since enlarging $\mathbf{X}_j$'s column space can only improve the fit to the same $\mathbf{y}$. Average training MSE decreases monotonically to about 0.79, while average test MSE dips to a minimum near $j = 6$ before rising as noise-fitting predictors are added.

Part b

```

p_j <- j_grid + 1
B2 <- sapply(j_grid, function(j) {
  Xj <- cbind(1, X_all[, seq_len(j), drop = FALSE])
  Hj <- Xj %*% solve(crossprod(Xj)) %*% t(Xj)
  sum(((diag(n) - Hj) %*% mu)^2)
})
theory_train_mse <- B2 / n + 1 - p_j / n
theory_test_mse <- B2 / n + 1 + p_j / n

plot(
  j_grid, avg_train_mse, type = "l", lwd = 2, col = "#1F5A94",
  ylim = range(avg_train_mse, avg_test_mse, theory_train_mse, theory_test_mse),
  xlab = "Number of predictors (j)", ylab = "MSE"
)
lines(j_grid, avg_test_mse, lwd = 2, col = "#C84A16")
lines(j_grid, theory_train_mse, lwd = 2, lty = 2, col = "#1F5A94")
lines(j_grid, theory_test_mse, lwd = 2, lty = 2, col = "#C84A16")
legend(
  "topright", bty = "n", lwd = 2, lty = c(1, 1, 2, 2),
  col = c("#1F5A94", "#C84A16", "#1F5A94", "#C84A16"),
  legend = c(
    "Average training MSE", "Average test MSE",
    "Theoretical E(MSE_train)", "Theoretical E(MSE_test)"
  )
)

```

```
)
)
```

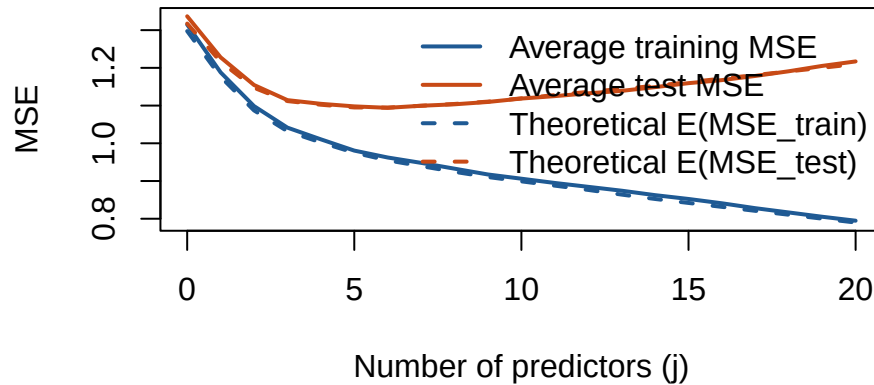


Figure 2: Simulation averages vs. theoretical expectations.

```
print(c(
  max_abs_diff_train = max(abs(avg_train_mse - theory_train_mse)),
  max_abs_diff_test = max(abs(avg_test_mse - theory_test_mse))
))
```

```
max_abs_diff_train  max_abs_diff_test
0.01723885         0.01929096
```

The theoretical curves track the simulation averages closely (max discrepancy ≈ 0.02), confirming that the bias term B_j^2/n (shrinking in j) trades off against the parameter-count term p_j/n , which penalizes training MSE and inflates test MSE.

Part c

```
one_run_test_mse <- mse_test[1, ]

plot(
  j_grid, one_run_test_mse, type = "l", lwd = 1.5, col = "grey55",
  ylim = range(one_run_test_mse, avg_test_mse, theory_test_mse),
  xlab = "Number of predictors (j)", ylab = "Test MSE"
)
lines(j_grid, avg_test_mse, lwd = 2, col = "#C84A16")
lines(j_grid, theory_test_mse, lwd = 2, lty = 2, col = "#C84A16")
legend(
  "topright", bty = "n", lwd = c(1.5, 2, 2), lty = c(1, 1, 2),
  col = c("grey55", "#C84A16", "#C84A16"),
  legend = c("Test MSE (one run)", "Average test MSE (200 runs)", "Theoretical E(MSE_test)")
)
```

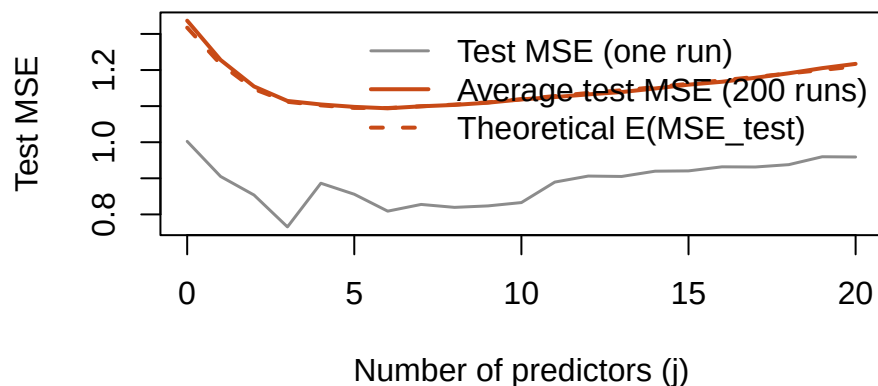


Figure 3: One simulation run vs. average test MSE.

The single-run curve is rougher because it reflects one noise draw, while the average curve cancels sampling noise across 200 runs. Expected test MSE decomposes into bias B_j^2/n (non-increasing in j) and variance p_j/n (increasing in j). Training MSE reflects only the bias side — it decreases monotonically regardless of whether added predictors are real signal — so choosing j by training MSE alone would always pick the largest model, missing the variance cost that makes an intermediate j optimal for test MSE.

Question 2 Answers

Part a

```
n <- 100
q <- 6
sigma2 <- 1
X_all <- outer(
  1:n, 1:q,
  function(i, k) sqrt(2) * cos(pi * k * (i - 0.5) / n)
)
beta <- rep(0.5, q)
x0 <- c(0, 1, 0, 0, 1, 0)
mu0 <- sum(x0 * beta)

theory_error <- sapply(0:q, function(j) {
  bias <- if (j < q) sum(x0[(j + 1):q] * beta[(j + 1):q]) else 0
  variance_term <- sigma2 / n * (1 + if (j > 0) sum(x0[1:j]^2) else 0)
  bias^2 + variance_term
})

print(mu0)
```

```
[1] 1
```

```
print(round(theory_error, 4))
```

```
[1] 1.01 1.01 0.27 0.27 0.27 0.03 0.03
```

$\mu_0 = 1$. The error is 1.01 at $j = 0, 1$; 0.27 at $j = 2, 3, 4$; 0.03 at $j = 5, 6$ — it changes only when predictors 2 and 5 enter, since predictors 1, 3, 4, 6 have $x_{0k} = 0$ and affect neither the bias nor variance term at this point.

Part b

```
set.seed(43203)
n_sim <- 200
squared_error <- matrix(NA, n_sim, q + 1)

for (s in 1:n_sim) {
  epsilon <- rnorm(n, sd = sqrt(sigma2))
  y <- as.vector(X_all %*% beta + epsilon)

  for (j in 0:q) {
    Xj <- cbind(1, X_all[, seq_len(j), drop = FALSE])
    fit <- lm.fit(x = Xj, y = y)
    x0j <- c(1, x0[seq_len(j)])
    mu0_hat <- sum(x0j * fit$coefficients)
    squared_error[s, j + 1] <- (mu0_hat - mu0)^2
  }
}

avg_squared_error <- colMeans(squared_error)

plot(
  0:q, avg_squared_error, type = "b", pch = 16, lwd = 2, col = "#1F5A94",
  ylim = range(avg_squared_error, theory_error),
  xlab = "Number of predictors (j)", ylab = "Squared error at x0"
)
lines(0:q, theory_error, type = "b", pch = 17, lwd = 2, lty = 2, col = "#C84A16")
legend(
  "topright", bty = "n", lwd = 2, lty = c(1, 2), pch = c(16, 17),
  col = c("#1F5A94", "#C84A16"),
  legend = c("Simulation average (200 runs)", "Theoretical expectation")
)
```

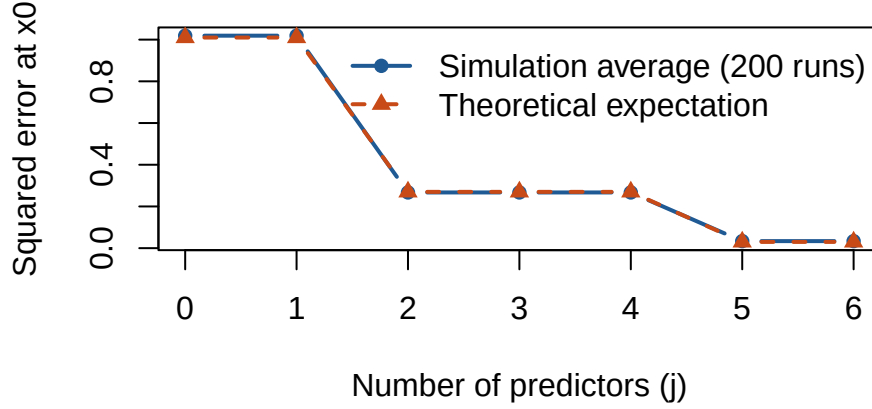


Figure 4: Simulated average vs. theoretical squared error at $\mathbf{x}_0$.

```
print(which.min(theory_error) - 1)
```

[1] 5

The simulation matches the theoretical curve closely; the smallest j minimizing the theoretical error is $j = 5$.

Part c

$\mu_0 = \sum_k x_{0k}\beta_k$, and $\mathbf{x}_0$ is zero except at positions 2 and 5, so terms for $k = 1, 3, 4, 6$ vanish at this point regardless of their β_k — those predictors matter elsewhere, not at $\mathbf{x}_0$. This contrasts with Question 1's test MSE, averaged over all rows of $\mathbf{X}_{\text{all}}$: there every nonzero- β_k predictor contributes on average, since no coordinate is uniformly zero across rows, whereas a single-point target's error depends only on the coordinates nonzero at that one point.

Question 3 Answers

Part a

Write $y = \mu + \epsilon$, so $\mathbf{Hy} = \mathbf{H}\mu + \mathbf{H}\epsilon$ and, using $\mathbf{H}^\top = \mathbf{H}$, $\mathbf{H}^2 = \mathbf{H}$,

$$\mathbf{y} - \mathbf{Hy} = (\mathbf{I}_n - \mathbf{H})\mu + (\mathbf{I}_n - \mathbf{H})\epsilon.$$

Squaring the norm and using $(\mathbf{I}_n - \mathbf{H})^\top(\mathbf{I}_n - \mathbf{H}) = \mathbf{I}_n - \mathbf{H}$,

$$\|\mathbf{y} - \mathbf{Hy}\|_2^2 = B^2 + 2\mu^\top(\mathbf{I}_n - \mathbf{H})\epsilon + \epsilon^\top(\mathbf{I}_n - \mathbf{H})\epsilon.$$

Conditional on $\mathbf{X}$, $E(\epsilon \mid \mathbf{X}) = \mathbf{0}$, so the cross term has zero expectation, and by the trace identity with $\mathbf{A} = \mathbf{I}_n - \mathbf{H}$,

$$E(\epsilon^\top(\mathbf{I}_n - \mathbf{H})\epsilon \mid \mathbf{X}) = \sigma^2 \text{tr}(\mathbf{I}_n - \mathbf{H}) = \sigma^2(n - p).$$

Dividing by n gives $E(\text{MSE}_{\text{train}} \mid \mathbf{X}) = B^2/n + \sigma^2(1 - p/n)$.

For the test error, $\mathbf{y}^* - \mathbf{H}\mathbf{y} = (\mathbf{I}_n - \mathbf{H})\boldsymbol{\mu} + \boldsymbol{\epsilon}^* - \mathbf{H}\boldsymbol{\epsilon}$. Since $\boldsymbol{\epsilon}^*$ and $\boldsymbol{\epsilon}$ are independent and both mean zero given $\mathbf{X}$, every cross term in the expanded squared norm has zero expectation, leaving

$$E(\|\mathbf{y}^* - \mathbf{H}\mathbf{y}\|_2^2 \mid \mathbf{X}) = B^2 + E(\boldsymbol{\epsilon}^{*\top}\boldsymbol{\epsilon}^* \mid \mathbf{X}) + E(\boldsymbol{\epsilon}^\top \mathbf{H}^\top \mathbf{H} \boldsymbol{\epsilon} \mid \mathbf{X}) = B^2 + \sigma^2 n + \sigma^2 p,$$

using $\text{tr}(\mathbf{I}_n) = n$ and $\mathbf{H}^\top \mathbf{H} = \mathbf{H}$ with $\text{tr}(\mathbf{H}) = p$. Dividing by n gives $E(\text{MSE}_{\text{test}} \mid \mathbf{X}) = B^2/n + \sigma^2(1 + p/n)$.

Part b

The bias terms cancel, leaving the expected optimism

$$E(\text{MSE}_{\text{test}} - \text{MSE}_{\text{train}} \mid \mathbf{X}) = \sigma^2 \left(1 + \frac{p}{n}\right) - \sigma^2 \left(1 - \frac{p}{n}\right) = \frac{2\sigma^2 p}{n}.$$

```
n <- 120
p <- 5
bias_per_n <- 0.04
sigma2 <- 1

expected_train_mse <- bias_per_n + sigma2 * (1 - p / n)
expected_test_mse <- bias_per_n + sigma2 * (1 + p / n)
expected_optimism <- expected_test_mse - expected_train_mse

print(c(
  expected_train_mse = expected_train_mse,
  expected_test_mse = expected_test_mse,
  expected_optimism = expected_optimism
))
```

```
expected_train_mse  expected_test_mse  expected_optimism
      0.99833333      1.08166667      0.08333333
```

Expected train MSE ≈ 0.998 , test MSE ≈ 1.082 , optimism ≈ 0.083 (matching $2\sigma^2 p/n$). This holds only in expectation over $\boldsymbol{\epsilon}, \boldsymbol{\epsilon}^*$; for a single realized pair, a small $\boldsymbol{\epsilon}^*$ or large $\boldsymbol{\epsilon}$ can flip the inequality even though it holds on average.

Part c

```
n <- 120
p <- 5
rss <- 116.4
sigma2_hat <- 0.96

mse_test_hat <- (rss + 2 * p * sigma2_hat) / n
Cp <- rss / sigma2_hat - n + 2 * p

print(c(mse_test_hat = mse_test_hat, Cp = Cp))
```

```
mse_test_hat      Cp
      1.05      11.25
```

```
print(sigma2_hat * (Cp + n) / n)
```

```
[1] 1.05
```

$\widehat{\text{MSE}}_{\text{test}} = 1.05$ and $C_p = 11.25$. Algebraically, $C_p = \text{RSS} / \hat{\sigma}^2 - n + 2p$ implies $\text{RSS} = \hat{\sigma}^2(C_p + n - 2p)$, so

$$\widehat{\text{MSE}}_{\text{test}} = \frac{\text{RSS} + 2p\hat{\sigma}^2}{n} = \frac{\hat{\sigma}^2(C_p + n)}{n} = \hat{\sigma}^2 \left(1 + \frac{C_p}{n} \right),$$

which the code confirms $(0.96(11.25 + 120)/120 = 1.05)$. Since $n, \hat{\sigma}^2$ are common across models, $\widehat{\text{MSE}}_{\text{test}}$ is an increasing affine function of C_p , so minimizing either gives the same ordering and the same minimizer.

Question 4 Answers

Part a

```
diabetes <- read.csv("data/diabetes.csv")
train <- diabetes[1:370, ]
n <- 370

fit_A <- lm(y ~ bmi + bp + s5 + sex + s1 + s2 + s4, data = train)
fit_B <- lm(y ~ bmi + bp + s5 + sex + s1 + s2 + s4 + s6, data = train)
fit_full <- lm(y ~ ., data = train)

model_summary <- data.frame(
  model = c("A", "B", "Full reference"),
  p = c(length(coef(fit_A)), length(coef(fit_B)), length(coef(fit_full))),
  RSS = c(sum(resid(fit_A)^2), sum(resid(fit_B)^2), sum(resid(fit_full)^2))
)
df_full <- n - model_summary$p[3]
sigma2_full <- model_summary$RSS[3] / df_full

print(model_summary)
```

	model	p	RSS
1	A	8	1098909
2	B	9	1089717
3	Full reference	11	1089452

```
print(c(residual_df_full = df_full, sigma2_full = sigma2_full))
```

residual_df_full	sigma2_full
359.000	3034.684

Model A has $p = 8$ and $\text{RSS} \approx 1,098,909$; Model B has $p = 9$ and $\text{RSS} \approx 1,089,717$; the full reference model has $p = 11$ and $\text{RSS} \approx 1,089,452$, with residual degrees of freedom $n - 11 = 359$ and $\hat{\sigma}_{\text{full}}^2 \approx 3,034.7$.

Part b

```
Cp_value <- function(rss, p) rss / sigma2_full - n + 2 * p
aic_reduced <- function(rss, p) n * log(rss / n) + 2 * p
bic_reduced <- function(rss, p) n * log(rss / n) + p * log(n)

criteria <- data.frame(
  model = c("A", "B"),
  p = model_summary$p[1:2],
  RSS = model_summary$RSS[1:2]
)
```

```

criteria$Cp <- Cp_value(criteria$RSS, criteria$p)
criteria$AIC_reduced <- aic_reduced(criteria$RSS, criteria$p)
criteria$BIC_reduced <- bic_reduced(criteria$RSS, criteria$p)

print(criteria)

```

	model	p	RSS	Cp	AIC_reduced	BIC_reduced
1	A	8	1098909	8.116253	2974.640	3005.948
2	B	9	1089717	7.087434	2973.532	3008.754

C_p (7.09 vs. 8.12) and reduced AIC (2973.5 vs. 2974.6) both favor Model B; reduced BIC (3005.9 vs. 3008.8) favors Model A. We compare only within each criterion, not across them.

Part c

Adding `s6` drops RSS by about 9,192 for one extra parameter — enough to beat C_p /AIC's effective penalty ($\approx 2\hat{\sigma}_{\text{full}}^2 \approx 6,070$), so both favor Model B. BIC's heavier penalty, $\log(370) \approx 5.91$ per parameter, outweighs the same improvement, favoring Model A. The disagreement just reflects different complexity penalties, not a contradiction.

Selecting the best-scoring candidate only shows it has the best fit-complexity trade-off *among those considered* — it does not verify the true data-generating process, which may not even be linear in these predictors; a selected model is a useful approximation, not proof of truth.

Question 5 Answers

Part a

```

diabetes <- read.csv("data/diabetes.csv")
train <- diabetes[1:370, ]
test <- diabetes[371:442, ]
predictor_order <- c("age", "sex", "bmi", "bp", "s1", "s2", "s3", "s4", "s5", "s6")

fit_and_test_mse <- function(train_data, eval_data, j) {
  if (j == 0) {
    fit <- lm(y ~ 1, data = train_data)
  } else {
    predictors <- paste(predictor_order[seq_len(j)], collapse = " + ")
    fit <- lm(as.formula(paste("y ~", predictors)), data = train_data)
  }
  mean((eval_data$y - predict(fit, newdata = eval_data))^2)
}

final_test_mse <- sapply(0:10, function(j) fit_and_test_mse(train, test, j))
names(final_test_mse) <- 0:10

print(round(final_test_mse, 1))

```

	0	1	2	3	4	5	6	7	8	9	10
	5822.9	5527.8	5527.2	3719.2	3246.8	3234.9	3207.6	2636.5	2624.9	2369.3	2484.8

```

print(which.min(final_test_mse) - 1)

```

9
9

Steps 2–3 misuse the test data: minimizing test MSE over eleven models uses that data to make a modeling decision, not just to evaluate an already-fixed model (here selecting $j = 9$, $\text{MSE} \approx 2,369$). Since eleven noisy test-MSE estimates come from the same 72 observations, the minimum is partly chosen *because* it looks good on that set — the same optimism-of-selection effect as Question 3, now applied to model selection. The reported minimum therefore understates the true future prediction error, even though no coefficients were fit on the test data.

Part b

```
set.seed(43205)
K <- 5
folds <- sample(rep(1:K, length.out = nrow(train)))

cv_mse <- sapply(0:10, function(j) {
  mean(sapply(1:K, function(k) {
    fit_and_test_mse(train[folds != k, ], train[folds == k, ], j)
  }))
})
names(cv_mse) <- 0:10
j_star <- which.min(cv_mse) - 1

print(round(cv_mse, 1))
```

0	1	2	3	4	5	6	7	8	9	10
5960.9	5809.3	5858.9	3989.4	3749.0	3789.1	3799.0	3198.3	3214.4	3104.1	3132.7

```
print(j_star)

9
9

final_fit <- if (j_star == 0) {
  lm(y ~ 1, data = train)
} else {
  predictors <- paste(predictor_order[seq_len(j_star)], collapse = " + ")
  lm(as.formula(paste("y ~", predictors)), data = train)
}
honest_test_mse <- mean((test$y - predict(final_fit, newdata = test))^2)
print(honest_test_mse)
```

```
[1] 2369.344
```

Split the 370 training rows into 5 folds; for each j , fit on 4 folds and evaluate on the held-out fold, averaging over folds for a CV MSE. Select j^* minimizing CV MSE ($j^* = 9$), then refit that model once on all 370 training rows. Only then is the final test set touched, once, to report a single honest MSE ($\approx 2,369$).

Part c

The eleven test-MSE values can still be reported transparently as an exploratory look at model complexity, but not as an unbiased estimate of future error, since the test set has already informed model choice. A valid final evaluation needs a genuinely new set of observations, never touched during fitting, CV selection, or this inspection, to score the now-fixed model once.